Name of Student : Raghav Mahesh Sarda	**PRN:** 25070122***235***
Subject: Programming with Java	**Semester: III**
ASSIGNMENT No : 6	**Year: AY 2026-27**
TITLE : Inner and Anonymous Classes	

Problem Statement

Write a Java program to demonstrate the use of an inner class, an anonymous class.

Learning Objectives

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Program:

Vehicle.java

```
// Assignment 6 - Exercise 1: Vehicle Application using Inner and
Anonymous Classes
import java.util.*;
class VehicleAction // Base class for anonymous class
{
   void performAction() // Method to be overridden
   {
        System.out.println("Vehicle default action started."); // Default
print
   }
}
public class Vehicle
{
   String brand; // Outer class variable
   class VehicleDetails // Inner class
   {
       void display() // Inner class method
       {
            System.out.println("Registered Vehicle Brand: " + brand); //
Accessing outer class variable
       }
   }
```

```
    public static void main(String args[]) // Main Method
    {
        Scanner sc = new Scanner(System.in);
        Vehicle v = new Vehicle(); // Outer class instance

        System.out.print("Enter Vehicle Brand: ");
        v.brand = sc.nextLine();

            Vehicle.VehicleDetails details = v.new VehicleDetails(); //
Instantiating inner class
        details.display(); // Calling inner class method

         VehicleAction action = new VehicleAction() // Creating anonymous
class
        {
            void performAction() // Overriding base method
            {
                    System.out.println("Action: Vehicle engine started and
ready to drive!");
            }
        };
        action.performAction(); // Executing anonymous class method
        sc.close();
    }
}
```

FoodDelivery.java

```
// Assignment 6 - Exercise 2: Food Delivery using Inner and
Anonymous Classes
import java.util.*;
class StatusTracker // Base class for anonymous class
{
    void updateStatus() // Method to be overridden
    {
        System.out.println("Status: Pending..."); // Default status
    }
}
public class FoodDelivery
{
    class OrderDetails // Inner class
    {
```

```
        String item; // Inner class variable for food item
        double price; // Inner class variable for price

        void display() // Method to display order
        {
             System.out.println("\nOrder Confirmation: " + item + " |
Total: Rs. " + price);
        }
    }

    public static void main(String args[]) // Main Method
    {
        Scanner sc = new Scanner(System.in);
            FoodDelivery app = new FoodDelivery(); // Outer class
instance
         FoodDelivery.OrderDetails order = app.new OrderDetails(); //
Inner class instance

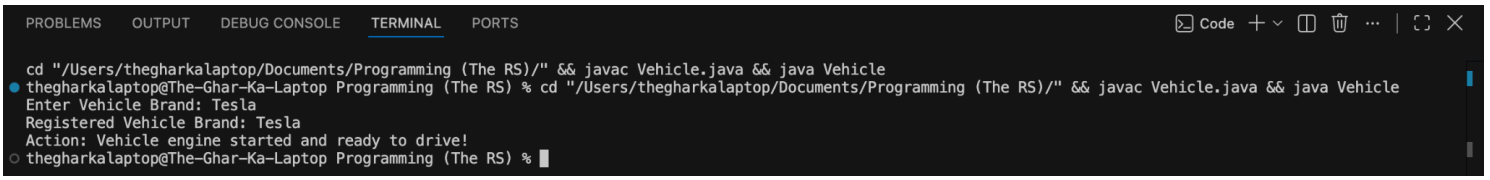
        System.out.print("Enter Food Item: ");
          order.item = sc.nextLine(); // Assigning item directly to
inner class variable
        System.out.print("Enter Price: ");
        order.price = sc.nextDouble(); // Assigning price directly to
inner class variable

        order.display(); // Showing order details

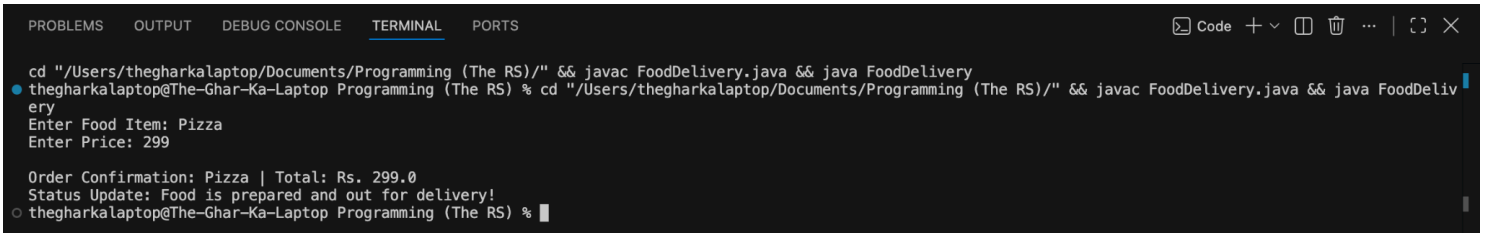
            StatusTracker status = new StatusTracker() // Creating
anonymous class for tracking
        {
            void updateStatus() // Overriding tracker method
            {
                 System.out.println("Status Update: Food is prepared
and out for delivery!");
            }
        };
        status.updateStatus(); // Executing anonymous class method
        sc.close();
    }
}
```

Output:

Vehicle.java



FoodDelivery.java



Conclusion:

In this assignment, the concepts of advanced class design and event-driven data handling using inner and anonymous classes were successfully implemented and verified. Inner classes provided strong encapsulation by logically grouping dependent structures within a host class, and anonymous classes allowed quick on-the-fly method overriding without the need to define separate concrete class files.